In [1]:
```
#Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import sqlite3
```

In [2]:
```
#Import data
conn = sqlite3.connect("C:/Users/admin/Downloads/Churn analysis/customer_churn.db")
sql_query="select name from sqlite_master where type='table';"

tables=pd.read_sql(sql_query, conn)
# print("Tables found in database:", tables)

#create dataframe
for table_name in tables['name']:
    df = pd.read_sql(f"select * from {table_name}", conn)
    globals()[f"df_{table_name}"]=df
    print(f"DataFrame Created: df_{table_name}")
conn.close()
```

```
DataFrame Created: df_db_customer
DataFrame Created: df_db_subscription
DataFrame Created: df_db_support
```

In [3]:
```
df.head()
```

Out[3]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

In [4]:
```
#First Table
df_db_customer.head()
```

Out[4]:

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

Data Cleaning

Table 1 = df_db_customer

In [5]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   customerid  21 non-null     object
 1   name        21 non-null     object
 2   country     18 non-null     object
 3   state       21 non-null     object
 4   gender      21 non-null     object
 5   dob         21 non-null     object
 6   interests   4 non-null      object
 7   pincode     0 non-null      object
dtypes: object(8)
memory usage: 1.4+ KB
```

In [6]:
```
#Droping empty or not usefull column
df_db_customer.drop(columns=['interests', 'pincode'],axis=1, inplace=True)
df_db_customer.head()
```

Out[6]:

	customerid	name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00

In [7]:
```
#Renaming Name
df_db_customer.rename(columns={'name':'customer_name '},inplace=True)
```

In [8]:
```
#Changing datatype of DoB
df_db_customer['dob']=pd.to_datetime(df_db_customer['dob'])
df_db_customer.head()
```

Out[8]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05

In [9]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype         
---  ------          --------------  -----         
 0   customerid      21 non-null     object        
 1   customer_name   21 non-null     object        
 2   country         18 non-null     object        
 3   state           21 non-null     object        
 4   gender          21 non-null     object        
 5   dob             21 non-null     datetime64[ns]
dtypes: datetime64[ns](1), object(5)
memory usage: 1.1+ KB
```

In [10]:
```
df_db_customer['gender'].unique()
```

Out[10]:
```
array(['Male', 'Female', 'Women', 'Men'], dtype=object)
```

In [11]:
```
#Standarddizing Gender
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men':'Male', 'Women':
```

In [12]:
```
df_db_customer['gender'].unique()
```

Out[12]:
```
array(['Male', 'Female'], dtype=object)
```

In [13]:
```
df_db_customer[df_db_customer['country'].isnull()]
```

Out[13]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	None	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	None	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01

In [14]:
```
#missing rows-> country mapping state-> filling
mapping_country_state=df_db_customer.dropna(subset=['country']).set_index('state')[
df_db_customer['country']=df_db_customer['country'].fillna(df_db_customer['state'].
```

In [15]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      21 non-null     object
 1   customer_name   21 non-null     object
 2   country         21 non-null     object
 3   state           21 non-null     object
 4   gender          21 non-null     object
 5   dob             21 non-null     datetime64[ns]
dtypes: datetime64[ns](1), object(5)
memory usage: 1.1+ KB
```

Table2 = df_db_subscription

In [16]:
```
#2nd Table
df_db_subscription.head()
```

Out[16]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [17]:
```
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     object
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      object
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

In [18]:
```
date_col=['subscription_start_date','renewal_date','cancellation_date']
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     datetime64[ns]
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     datetime64[ns]
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      datetime64[ns]
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB
```

Table3 = df_db_support

In [19]:
```
#3rd Table
df_db_support.head()
```

Out[19]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

In [20]:
```
#3rd Table
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64 
 4   col_1           0 non-null      object
 5   comment         4 non-null      object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

In [21]:
```
df_db_support['complaint_date']=pd.to_datetime(df_db_support['complaint_date'])
```

In [22]:
```
df_db_support.drop(columns=['col_1','comment'],inplace=True)
```

In [23]:
```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype         
---  ------          --------------  -----         
 0   customerid      9 non-null      object        
 1   complaint_date  9 non-null      datetime64[ns]
 2   escalations     9 non-null      object        
 3   csat_score      9 non-null      int64         
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 420.0+ bytes
```

In [24]:
```
#fixing duplicates
df_db_support['complaint_count']= df_db_support.groupby('customerid')['customerid']
df_db_support=df_db_support.sort_values('complaint_count').drop_duplicates('custome
```

Feature Engineering & Data Analysis

In [25]:
```
#Creating new column using existing column
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
```

In [26]:
```
df=(df_db_subscription
.merge(df_db_customer, on='customerid',how='left')
.merge(df_db_support, on='customerid',how='left'))
```

In [27]:
```
df.head(5)
```

Out[27]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

5 rows × 21 columns

In [28]:
```
df.shape
```

Out[28]:
```
(21, 21)
```

Data Analysis

In [29]:
```
df.to_csv('exported_churn_data_csv',index=False)
```

In [30]:
```
#churn_rate
churn_rate=df['churn_flag'].mean()*100
print('Churn_rate',round(churn_rate,2),'%')
```

```
Churn_rate 28.57 %
```

In [31]:
```
#retension_rate
retension_rate=100-df['churn_flag'].mean()*100
print('retension_rate',round(retension_rate,2),'%')
```

```
retension_rate 71.43 %
```

In [32]:
```
df.head()
```

Out[32]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

5 rows × 21 columns

In [33]:
```
churn_by_plan = df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).rese
churn_by_plan
```

Out[33]:

	plan_type	churn_rate_pcy
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

60% of our Basic user are leaving

In [34]:
```
#churn_by_state with monthly_loss and total_user
df_state = df.groupby(by='state').agg(
    {'churn_flag': lambda x: (x.mean() * (100)).round(2),
    'monthly_charges': lambda x: x[df.loc[x.index, 'churn_flag']==1].sum(),
    'customerid':'count'
    }).reset_index()
df_state= df_state.rename(columns={'churn_flag':'churn_flag_state','monthly_charges
df_state
```

Out[34]:

	state	churn_flag_state	monthly_loss_charges	user_by_state
0	Delhi	25.00	13.99	4
1	Karnataka	100.00	20.98	2
2	Kathmandu	0.00	0.00	2
3	Maharashtra	0.00	0.00	3
4	Meghalaya	66.67	21.98	3
5	Nagaland	0.00	0.00	1
6	Rajasthan	0.00	0.00	2
7	Telangana	50.00	16.99	2
8	Uttar Pradesh	0.00	0.00	2

In [35]:
```
#Churn by subscritption with monthly_loss and total_user
df_sub = df.groupby(by='subscription_type').agg(
    {
    'churn_flag': lambda x: (x.mean() * (100)).round(2),
    'monthly_charges': lambda x: x[df.loc[x.index, 'churn_flag']==1].sum(),
    'customerid':'count'
    }).reset_index()
df_sub= df_sub.rename(columns={'churn_flag':'churn_flag_state','monthly_charges':'m
df_sub
```

Out[35]:

	subscription_type	churn_flag_state	monthly_loss_charges	user_by_state
0	Organic	0.00	0.00	9
1	Paid	16.67	12.99	6
2	Refferal	83.33	60.95	6

In [36]:
```
#Average Revenue Per User AUPR
aupr=df['monthly_charges'].mean()
aupr
```

Out[36]:
```
np.float64(18.847142857142856)
```

In [37]:
```
#tenure by each cutomer
today = pd.Timestamp.today()
df['tenure_days']=np.where(
    df['cancellation_date'].notnull(),
    (df['cancellation_date']-df['subscription_start_date']).dt.days,
    (today-df['subscription_start_date']).dt.days
)
df['tenure_days'].head(3)
```

Out[37]:
```
0    2036.0
1    1501.0
2    1421.0
Name: tenure_days, dtype: float64
```

In [38]:
```
#avg tenure by cutomer
avg_tenure=df['tenure_days'].mean()
round(avg_tenure)
```

Out[38]:
```
1558
```

In [39]:
```
# Revenue at risk
revenue_loss=df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
revenue_loss
```

Out[39]:
```
np.float64(73.94)
```

In [40]:
```
#Escalation rate
escalation_rate=(df['escalations']=='Y').mean()*100
escalation_rate.round(2)
```

Out[40]:
```
np.float64(14.29)
```

In [41]:
```
#Avg complaint per user
avg_complaints=df['complaint_count'].sum()/df['customerid'].nunique()
avg_complaints.round(2)
```

Out[41]:
```
np.float64(0.43)
```

In [42]:
```
#correlation ecalaion vs churm
# df[['escalations','churn_flag']].dropna()
df['escalations']=np.where(df['escalations']=='Y',1,0) #encoding
correlation=df[['churn_flag','escalations']].corr()
correlation
```

Out[42]:

	churn_flag	escalations
churn_flag	1.000000	0.645497
escalations	0.645497	1.000000

In [43]:
```
#Creating new column using existing column
condition=[
    (df['churn_score'] <50),
    (df['churn_score'] >=50) & (df['churn_score']<70),
    (df['churn_score'] >70)
]
choice=['low', 'medium','high']
df['churn_risk']=np.select(condition,choice,default='unknown')
df.head(4)
```

Out[43]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A

4 rows × 23 columns

Visualization wit Matplotlib

In [44]:
```
df_visual = df.copy()
# pd.set_option('display.max_columns', None)
df_visual.head()
```

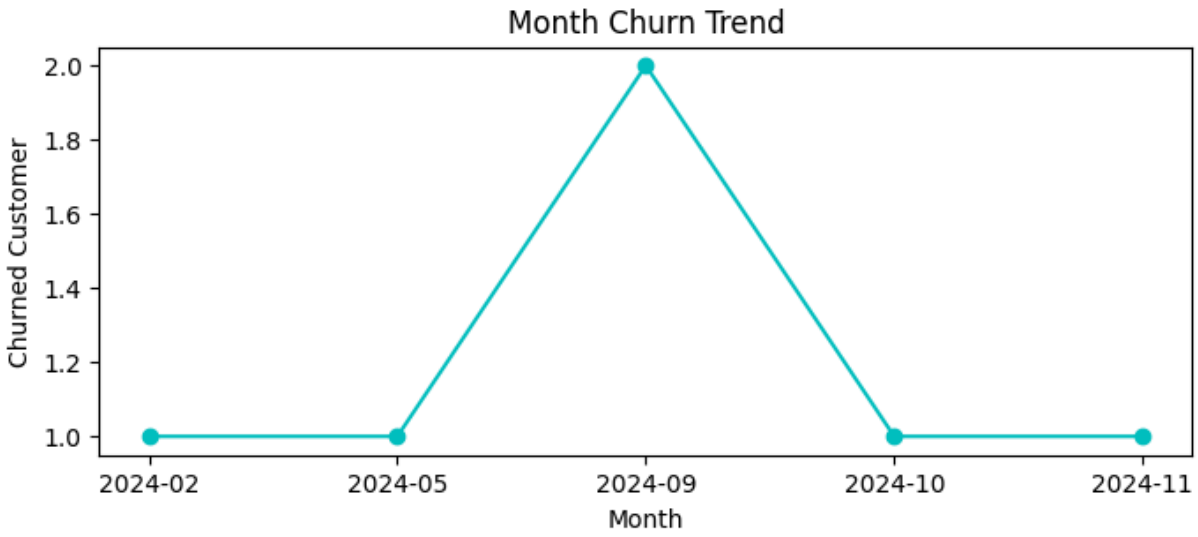
Out[44]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

5 rows × 23 columns

In [45]:
```
#churn by month
df_visual['cancelation_month']=df_visual['cancellation_date'].dt.to_period('M')
churn_trend=df_visual[df_visual['churn_flag']==1].groupby('cancelation_month').size
```

In [46]:
```
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='c',marker='o')
plt.title('Month Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customer')
plt.show()
```

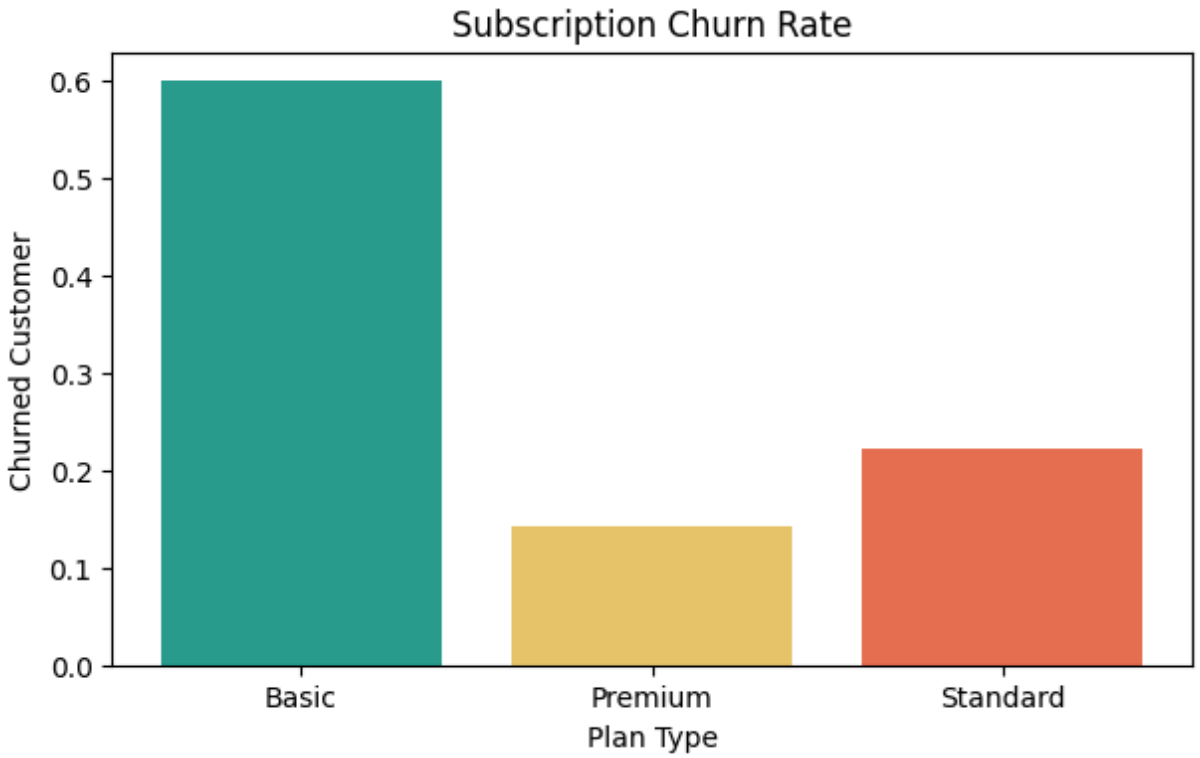


In [47]:
```
#Churn by subscription plan
sub_churn=df_visual.groupby('plan_type')['churn_flag'].mean()
sub_churn
```

Out[47]:
```
plan_type
Basic       0.600000
Premium     0.142857
Standard    0.222222
Name: churn_flag, dtype: float64
```

In [48]:
```
plt.figure(figsize=(7,4))
colors = ['#2A9D8F', '#E9C46A', '#E76F51']
plt.bar(sub_churn.index,sub_churn.values,color=colors)
plt.title('Subscription Churn Rate')
plt.xlabel('Plan Type')
plt.ylabel('Churned Customer')
```

Out[48]:
```
Text(0, 0.5, 'Churned Customer')
```

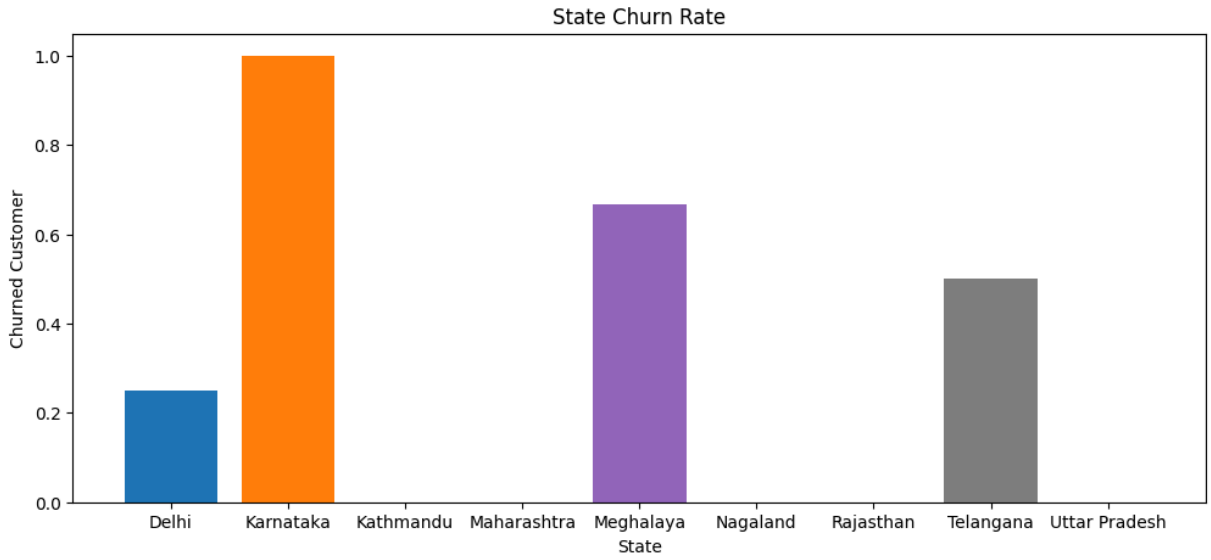


In [49]:
```
#churn by state
state_churn = df_visual.groupby('state')['churn_flag'].mean()
state_churn
```

Out[49]:
```
state
Delhi           0.250000
Karnataka       1.000000
Kathmandu       0.000000
Maharashtra     0.000000
Meghalaya       0.666667
Nagaland        0.000000
Rajasthan       0.000000
Telangana       0.500000
Uttar Pradesh   0.000000
Name: churn_flag, dtype: float64
```

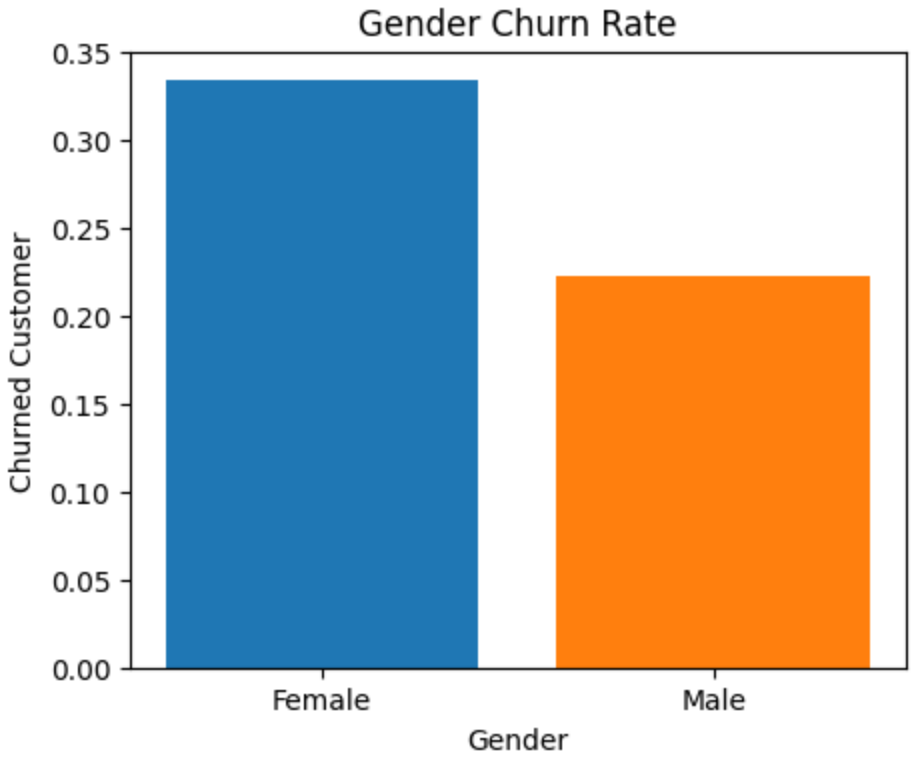
In [50]:
```
plt.figure(figsize=(12,5))
colors = [
    '#1F77B4','#FF7F0E','#2CA02C','#D62728','#9467BD','#8C564B','#E377C2','#7F7F7F'
]
plt.bar(state_churn.index,state_churn.values,color=colors)
plt.title('State Churn Rate')
plt.xlabel('State')
plt.ylabel('Churned Customer')
```

Out[50]:
```
Text(0, 0.5, 'Churned Customer')
```



In [51]:
```
#churn by gender
gender_churn = df_visual.groupby('gender')['churn_flag'].mean()
plt.figure(figsize=(5,4))
colors = ['#1F77B4','#FF7F0E']
plt.bar(gender_churn.index,gender_churn.values,color=colors)
plt.title('Gender Churn Rate')
plt.xlabel('Gender')
plt.ylabel('Churned Customer')
```

Out[51]:
```
Text(0, 0.5, 'Churned Customer')
```



Visualiztion with Seaboen

In [52]:
```
import warnings
warnings.filterwarnings('ignore')
```

Not the correct ways

In [53]:
```
#encoding but this way is wong
df_encode = df_visual[['plan_type', 'contract_type', 'monthly_charges', 'churn_scor
       'churn_flag', 'escalations',
       'tenure_days', 'churn_risk']]

df_categorical = ['plan_type', 'contract_type', 'churn_risk']

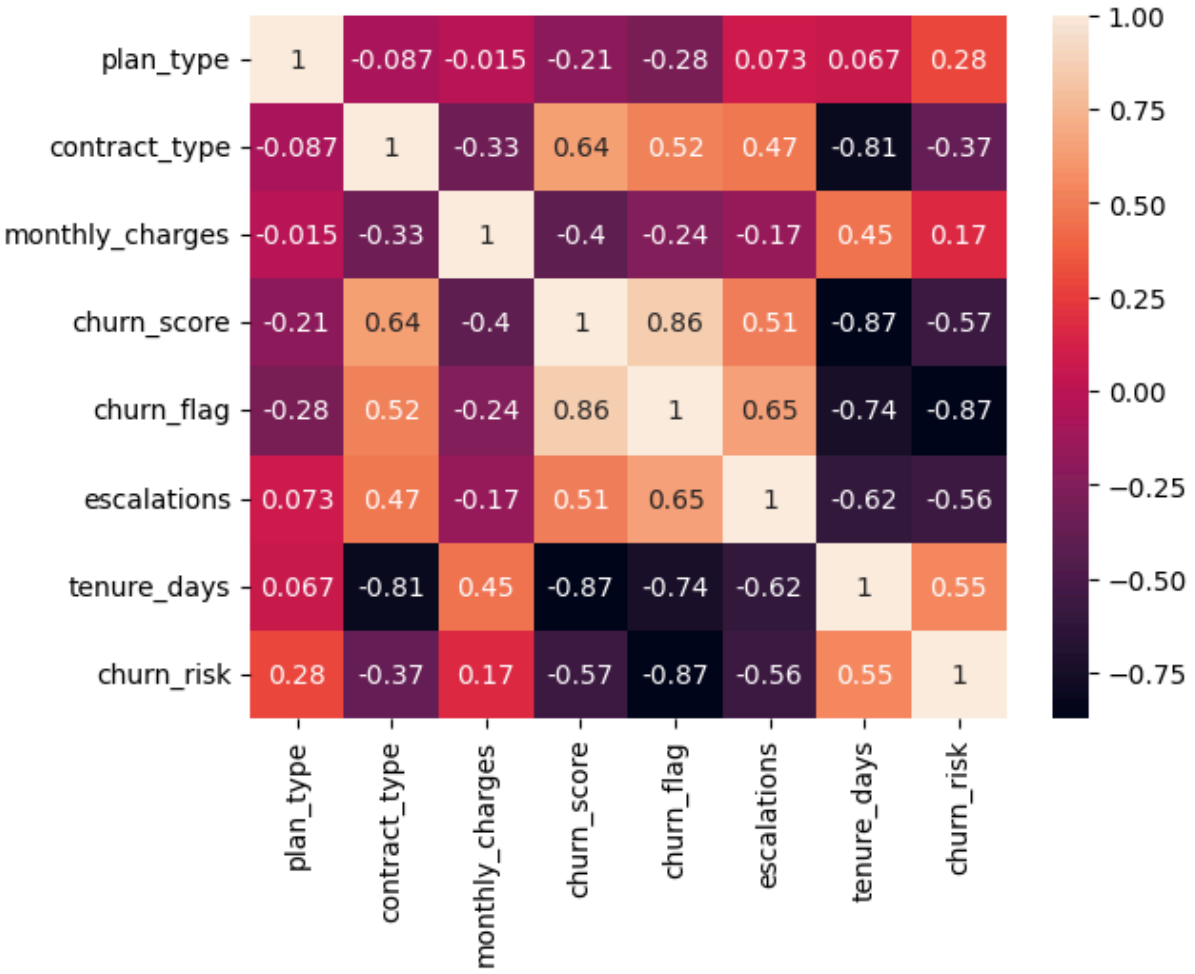
for col in df_categorical:
    df_encode[col] = df_encode[col].astype('category').cat.codes
df_encode.head()
```

Out[53]:

	plan_type	contract_type	monthly_charges	churn_score	churn_flag	escalations	tenure_
0	2	0	13.99	12	0	0	2(
1	1	0	12.99	91	1	0	1
2	0	1	6.99	34	0	0	1
3	1	0	22.99	8	0	0	2
4	2	1	13.99	88	1	1	

In [54]:
```
sns.heatmap(df_encode.corr(),annot=True)
```

Out[54]:
```
<Axes: >
```



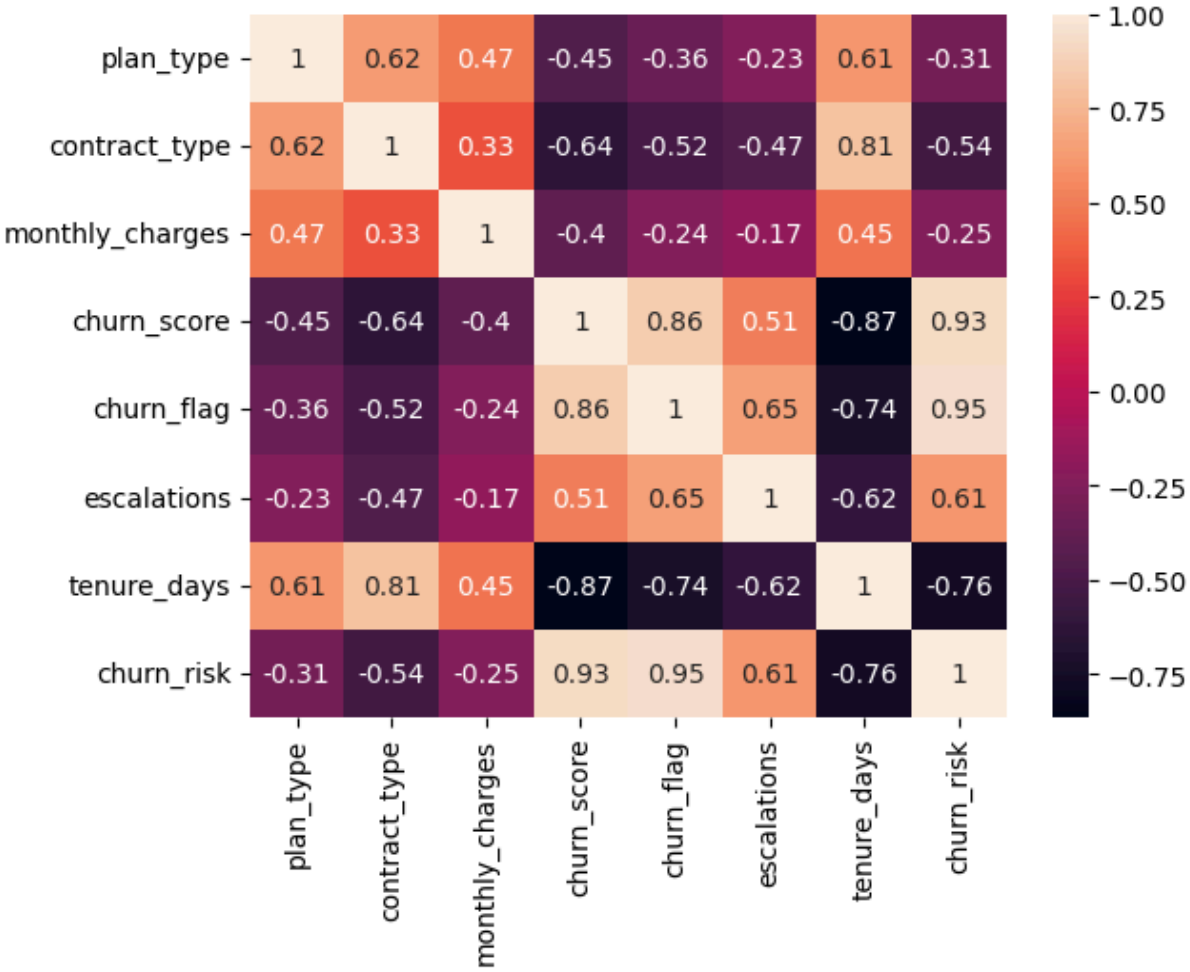
In [55]:
```
df_encode1=df_visual[['plan_type', 'contract_type', 'monthly_charges', 'churn_score
       'churn_flag','escalations','tenure_days', 'churn_risk']]
order_mapping= {'plan_type':['Basic','Standard','Premium'],
                'contract_type':['Monthly','Annual'],
                'churn_risk':['low','medium','high']
}
for col, order in order_mapping.items():
    custom_type = pd.CategoricalDtype(categories=order, ordered=True)
    df_encode1[col] = df_encode1[col].astype(custom_type).cat.codes
df_encode1.head()
```

Out[55]:

	plan_type	contract_type	monthly_charges	churn_score	churn_flag	escalations	tenure_
0	1	1	13.99	12	0	0	2
1	2	1	12.99	91	1	0	1
2	0	0	6.99	34	0	0	1
3	2	1	22.99	8	0	0	2
4	1	0	13.99	88	1	1	

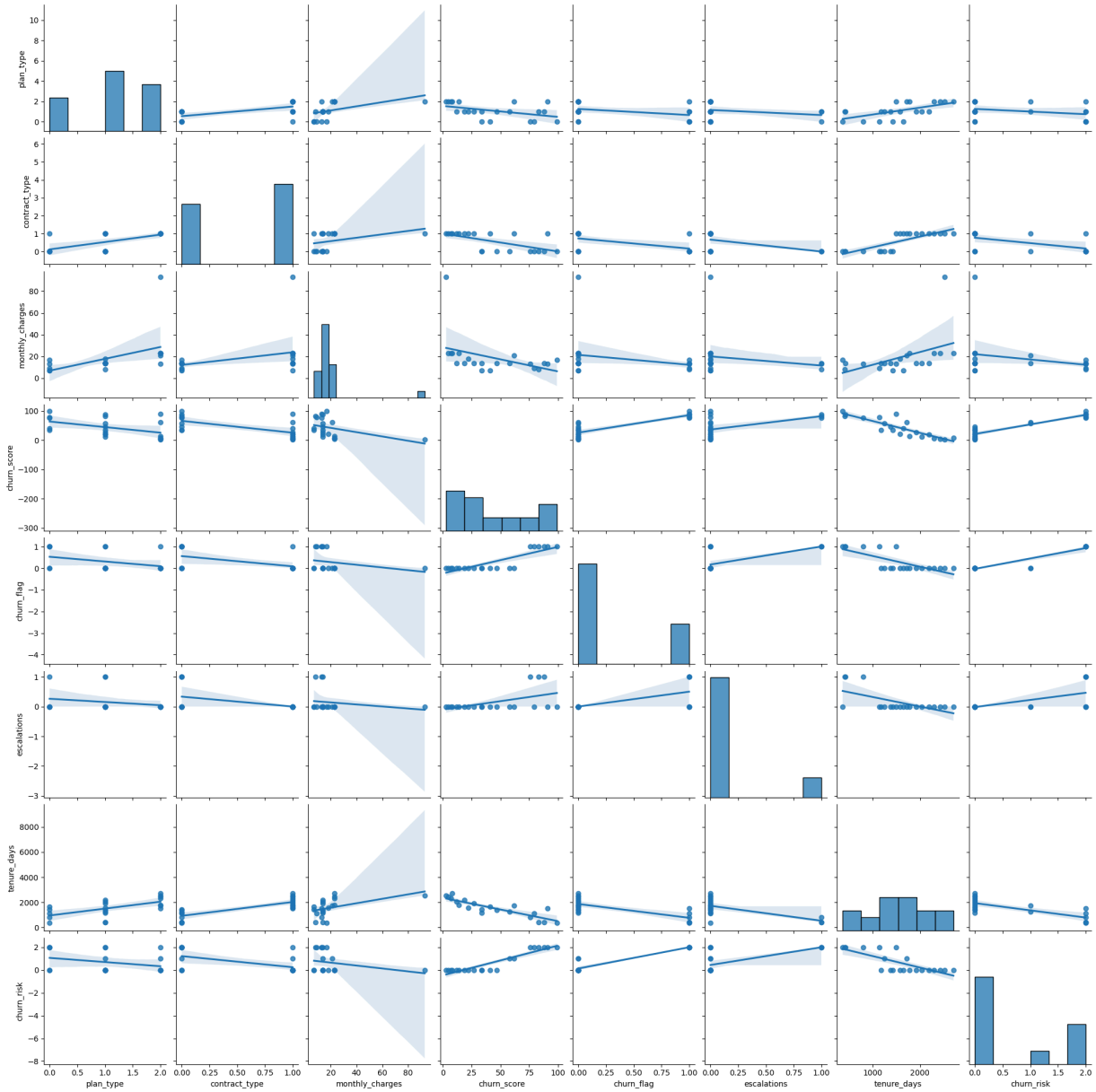
In [56]:
```
sns.heatmap(df_encode1.corr(),annot=True)
```

Out[56]: <Axes: >



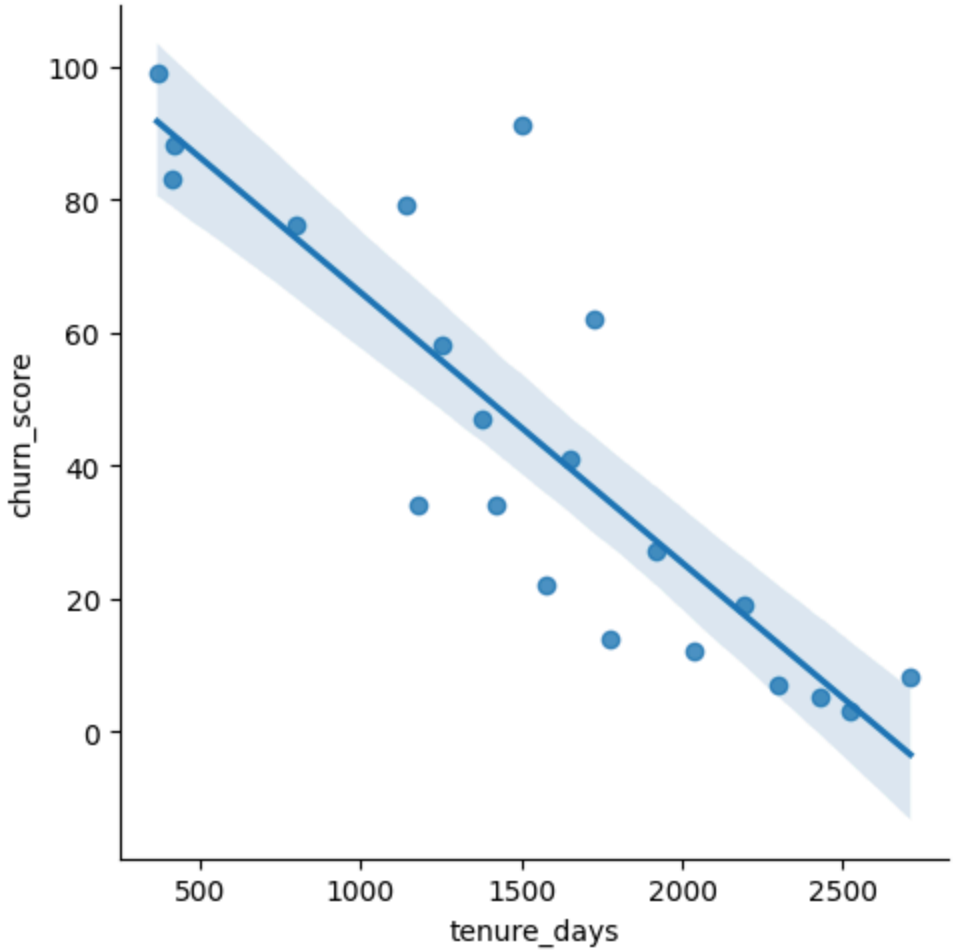
In [57]:
```
#Relatopnship in a dataset
sns.pairplot(df_encode1, kind='reg')
```

Out[57]: <seaborn.axisgrid.PairGrid at 0x18a0f5d4ec0>



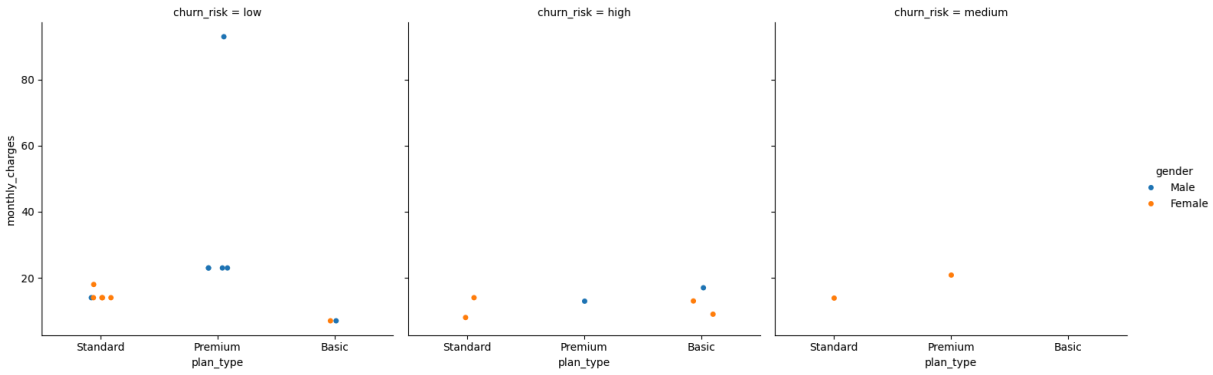
In [58]:
```
sns.lmplot(df_encode1,x='tenure_days',y="churn_score")
```

Out[58]:
```
<seaborn.axisgrid.FacetGrid at 0x18a0251a270>
```



In [59]:
```
#catplot/facegrid plot multi_d plot
sns.catplot(data=df_visual,x='plan_type',y='monthly_charges',hue='gender',col='chur
```

Out[59]: <seaborn.axisgrid.FacetGrid at 0x18a146666f0>



Pivot Table

In [62]:
```
pd.pivot_table(data=df_visual,index='plan_type', values='churn_flag',aggfunc='mean'
```

Out[62]:

	plan_type	churn_flag
0	Basic	0.600000
1	Premium	0.142857
2	Standard	0.222222

In [68]:
```
pd.pivot_table(data=df_visual,index='plan_type', columns='gender', values=['churn_f
               aggfunc={
                   'churn_flag':'mean',
                   'monthly_charges': 'sum',
                   'customerid': 'count'
               }).reset_index()
```

Out[68]:

	plan_type	churn_flag		customerid		monthly_charges	
gender		**Female**	**Male**	**Female**	**Male**	**Female**	**Male**
0	Basic	0.666667	0.500000	3	2	28.97	23.98
1	Premium	0.000000	0.166667	1	6	20.99	197.94
2	Standard	0.250000	0.000000	8	1	109.92	13.99

In [63]:
```
df_visual
```

Out[63]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contra
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	N
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	N
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	N
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	N
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	N
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	N
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	N
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	N

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contra
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	N

21 rows × 24 columns

Working with SQL in Python

In [69]:
```
#Creatin db in sql
conn = sqlite3.connect('database.sqlite')

#table detail
conn.execute('create table user (Name text, country text, income integer)')

#commit and save
conn.commit()
print('Run Succesfully')
```

```
Run Succesfully
```

In [73]:
```
#insert table
cursor = conn.cursor()
cursor.execute(
    """
    insert into user values
    ('Hassan', 'Pakistan', '50000'),
    ('Ali', 'Iran', '45000'),
    ('Sheraz', 'Pakistan', '35000'),
    ('Gillani', 'Afghanistan', '30000'),
    ('Abdullah', 'Saudi', '100000')
    """
)

conn.commit()
print('Run Succesfully')
```

```
Run Succesfully
```

In [78]:
```
# checking insert data
conn = sqlite3.connect('database.sqlite')
query = 'select * from User'

df_result = pd.read_sql(query, conn)
df_result
```

Out[78]:

	Name	country	income
0	Hassan	Pakistan	50000
1	Ali	Iran	45000
2	Sheraz	Pakistan	35000
3	Gillani	Afghanistan	30000
4	Abdullah	Saudi	100000

In [81]:

```
query = """
        select country, avg(income) as avg_income
        from user
        group by country
        """
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[81]:

	country	avg_income
0	Afghanistan	30000.0
1	Iran	45000.0
2	Pakistan	42500.0
3	Saudi	100000.0

In [82]:

```
conn.close()
```

In [ ]: